GUIA DE ESTUDOS

ENGENHARIA DE SOFTWARE

Entenda os conceitos, aprenda a ler e construir diagramas UML e pratique para a prova.

EXPLICAÇÕES COM ANALOGIAS • RESUMOS • VÍDEOS EM PORTUGUÊS • 50 EXERCÍCIOS

Material preparado a partir do PDF da disciplina enviado nesta conversa.
Complementos externos estão identificados nas referências.

Novembro de 2026

Como usar este material

A ideia é parar de decorar nomes soltos e começar a enxergar o sistema como uma história. Primeiro você entende o problema; depois escolhe qual desenho responde à pergunta; por fim, representa as informações com a notação adequada.

A ideia central da matéria

Engenharia de Software é organizar o desenvolvimento de um sistema para que ele resolva o problema certo, com qualidade, dentro das restrições do projeto. A UML é uma linguagem visual que ajuda a representar partes desse sistema. Ela não é o código e não é, sozinha, uma metodologia de desenvolvimento.

Ordem recomendada de estudo

1. Leia os fundamentos, os requisitos e os ciclos de vida para entender de onde vem a necessidade de modelar.
2. Aprenda primeiro os seis diagramas que aparecem com mais força nos exemplos e atividades do material: casos de uso, atividades, classes, objetos, sequência e máquina de estados.
3. Depois revise os outros oito diagramas da UML. Você precisa saber para que servem e como diferenciá-los, mesmo que não vá desenhar todos em profundidade.
4. Faça os exercícios 1 a 40 sem olhar o gabarito. Depois tente os exercícios 41 a 50 desenhando no papel ou em uma ferramenta.
5. Estude também a parte de métodos ágeis (Manifesto, Scrum, Kanban, XP, TDD, RAD e FDD), presente no final do PDF.
6. Use os vídeos apenas para reforçar um assunto que acabou de estudar. Não assista a várias aulas seguidas sem pausar para desenhar.

O mapa mental que resolve metade da prova

Pergunta que você quer responder	Diagrama / conceito
Quem usa o sistema e quais objetivos quer alcançar?	Casos de uso
Qual é o passo a passo de um processo?	Atividades
Quais são as coisas do domínio e como se relacionam?	Classes
Como os objetos estão preenchidos em um momento específico?	Objetos
Quem troca mensagens com quem e em qual ordem?	Sequência
Quais situações um objeto pode assumir ao longo da vida?	Máquina de estados
Como o sistema se divide em módulos ou contratos?	Pacotes / Componentes
Em quais máquinas e ambientes o software roda?	Implantação

Dica para a prova

Antes de desenhar, sublinhe no enunciado os substantivos (possíveis classes/objetos), os verbos (possíveis ações/casos de uso) e as mudanças de situação (possíveis estados). Depois pergunte: o professor quer mostrar estrutura, objetivo, fluxo, interação ou ciclo de vida?

1. Fundamentos de Engenharia de Software

O que é Engenharia de Software?

É a aplicação de práticas, métodos, processos e ferramentas para desenvolver, adquirir, testar, implantar e manter software de forma organizada, ética, responsável e eficiente. O foco não é simplesmente “fazer o programa funcionar”: é entregar uma solução que atenda às necessidades dos usuários, tenha qualidade e possa evoluir.

Analogia: construir uma casa

Programar é como executar a obra. Engenharia de Software começa antes: entender quem vai morar na casa, quantos cômodos precisa, quanto pode gastar, desenhar a planta, escolher materiais, organizar os profissionais, inspecionar a obra e planejar a manutenção. Uma casa não fica boa só porque o pedreiro trabalha rápido; um software também não fica bom só porque alguém codifica depressa.

Software é mais do que código

Software inclui programas e aplicativos, dados usados pelo sistema, telas e elementos visuais, documentação, manuais, arquivos de ajuda e tutoriais. O programa recebe informações, processa essas informações e produz um resultado, que frequentemente também é informação.

Teoria Geral dos Sistemas e visão sistêmica

A Teoria Geral dos Sistemas (TGS) procura entender um conjunto como um todo integrado, observando as relações entre suas partes. A visão sistêmica lembra que uma mudança em um componente pode afetar os demais.

Analogia: uma equipe de futebol

Não basta ter onze jogadores excelentes individualmente. Eles precisam se comunicar, ocupar posições e colaborar. Se o goleiro não recebe a informação de que a defesa saiu, o time inteiro pode sofrer. Em um sistema, interface, servidor, banco de dados e usuário também dependem uns dos outros.

Projeto de software: fatores que aumentam a chance de sucesso

- Objetivos claros: todos entendem o que será construído e qual resultado se espera.
- Envolvimento dos stakeholders: clientes, usuários, gestores e demais interessados participam das decisões.
- Planejamento: cronograma, custos, recursos e responsabilidades são definidos.
- Comunicação: reduz mal-entendidos e facilita decisões.
- Gerenciamento de riscos: problemas possíveis são identificados antes de comprometer o projeto.
- Monitoramento contínuo: o andamento é acompanhado e os desvios são corrigidos.

Arquitetura de software

É a estrutura organizacional do sistema: define seus componentes, as responsabilidades e interfaces desses componentes e a forma como se relacionam. É como a planta estrutural de um prédio; decisões arquiteturais importantes podem ser caras para alterar depois.

Ferramentas citadas no material

O material menciona IDEs, interfaces web/mobile/client, bancos de dados, fluxos, cronogramas, documentação e apresentações. Também cita ferramentas como Xcode, Android Studio, SQL Server, Visio, StarUML, Bizagi, WBS e Project. A ferramenta ajuda a produzir e organizar o trabalho, mas não substitui o entendimento do problema.

Base no material do professor: PDF enviado, pp. 1–10, 21–25, 34–35 e 70–82.

2. Ciclo de vida e construção de software

O que é ciclo de vida?

É a sequência organizada de etapas pelas quais o sistema passa, desde a concepção até a implantação, operação, manutenção e eventual descontinuação. O ciclo de vida descreve o processo de trabalho e ajuda a padronizar etapas e validações.

CONCEPÇÃO → DESENVOLVIMENTO → IMPLANTAÇÃO / OPERAÇÃO → MANUTENÇÃO → DESCONTINUAÇÃO

Analogia: preparar uma viagem

Você define o destino e o orçamento, planeja a rota, compra o que falta, faz a viagem, resolve imprevistos e depois avalia o que funcionou. O ciclo de vida organiza essas etapas. Os modelos de ciclo de vida são maneiras diferentes de organizar o caminho.

Modelos apresentados

Modelo	Como funciona	Analogia simples	Ponto forte / cuidado
Cascata	Etapas em sequência; uma fase é concluída antes da próxima.	Uma cachoeira: a água desce por etapas.	É previsível quando os requisitos são estáveis; mudanças tardias podem custar caro.
Prototipação	Cria-se um modelo inicial para descobrir e validar requisitos com o cliente.	Fazer um rascunho ou maquete antes da obra.	Ajuda quando o cliente sabe o objetivo, mas ainda não sabe detalhar as funções. O protótipo não deve ser confundido com o produto final.
Espiral / incremental	O desenvolvimento acontece em ciclos; cada incremento entrega uma parte funcional e há análise de riscos.	Montar a casa por etapas, verificando riscos a cada etapa.	Permite feedback e redução de riscos; exige experiência para analisar riscos e controlar as iterações.
RUP	Organiza o desenvolvimento orientado a objetos em quatro fases.	Uma jornada dividida em quatro grandes marcos.	Dá estrutura ao projeto e distribui atividades ao longo das fases.

RUP: memorize as quatro fases na ordem

Fase	O que acontece	Palavra-chave
1. Concepção / Iniciação	Comunicação com o cliente, escopo, riscos, custos, prazos, prioridades e requisitos preliminares.	Entender
2. Elaboração	Análise detalhada do domínio do problema e definição da forma básica da arquitetura.	Estruturar
3. Construção	Desenvolvimento e aquisição dos componentes; ocorre a maior parte da codificação.	Construir
4. Transição	Testes, versão beta, entrega, disponibilização e treinamento dos usuários.	Entregar

Pegadinha comum

Prototipação não significa necessariamente entregar o protótipo como sistema pronto. O protótipo serve para aprender e esclarecer requisitos. O material aponta o risco de o cliente querer usar como produto final algo feito rapidamente, sem considerar qualidade e manutenção.

Cinco princípios de construção

1. Minimizar a complexidade: preferir código simples de entender e manter.
2. Antecipar-se à mudança: preparar o sistema para evoluir sem quebrar sua estrutura principal.
3. Construir para verificação: facilitar testes, auditorias e identificação rápida de erros.
4. Reuso: reaproveitar rotinas já implementadas, como validação de CPF, em vez de reescrever sempre.
5. Padronização: adotar convenções de nomes, linguagens, métodos, ferramentas e modelagem.

Usabilidade, ergonomia e UX

Ergonomia trata de adequar o ambiente e a interface às pessoas. Usabilidade está ligada à facilidade de aprender, eficiência de uso, memorização, quantidade de erros e satisfação. UX (User Experience) trata da percepção e da resposta que resultam da interação com o produto.

Analogia: usar uma cozinha

Ergonomia: bancada na altura certa e utensílios ao alcance. Usabilidade: você aprende rápido onde ficam os objetos e consegue cozinhar sem errar. UX: além de funcionar, a experiência é agradável e não causa frustração.

Base no material do professor: PDF enviado, pp. 94–97, 100–127 e 137–143. Os slides intermediários do ciclo de vida são majoritariamente visuais; o resumo preserva os conceitos textuais disponíveis.

3. Requisitos de software

O que é requisito?

Requisito é a descrição de algo que o sistema deve fazer, de um serviço que oferece ou de uma restrição ao seu funcionamento. Os requisitos representam necessidades do cliente e dos usuários. Engenharia de Requisitos é o processo de descobrir, analisar, documentar e verificar esses requisitos.

Analogia: pedir um bolo

"Quero um bolo" é vago. Você precisa perguntar o sabor, tamanho, cobertura, prazo, restrições alimentares e orçamento. Se não perguntar, pode entregar um bolo bonito, mas completamente diferente do que a pessoa queria. Desenvolver sem entender requisitos causa retrabalho.

Requisitos funcionais e não funcionais

Tipo	Pergunta	Exemplos
Funcional (RF)	O que o sistema faz?	Cadastrar usuário; emitir relatório; calcular imposto; registrar empréstimo; enviar alerta.
Não funcional (RNF)	Como o sistema deve funcionar ou sob quais condições?	Responder em até 3 segundos; suportar 500 usuários; manter disponibilidade de 99,5%; criptografar dados; funcionar em dispositivos móveis.

Macete

Funcional = ação ou serviço. Não funcional = qualidade, desempenho, segurança, disponibilidade, usabilidade, compatibilidade ou restrição. Atenção: depois de detalhado, um requisito de segurança pode gerar funcionalidades concretas, como autenticação e autorização.

Como escrever um bom requisito

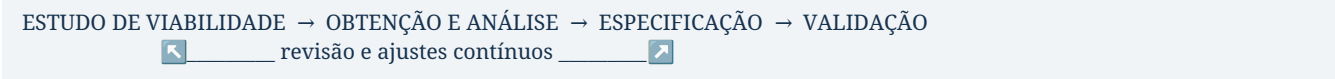
- Clareza: uma única interpretação possível; evite "adequado", "rápido" ou "fácil" sem explicação.
- Objetividade: vá direto ao ponto.
- Verificabilidade: deve ser possível testar ou validar de forma objetiva.
- Consistência: não deve contradizer outro requisito.

Exemplo: em vez de "o sistema deve ser rápido", escreva "o sistema deve responder às consultas de relatório em até 3 segundos, em condições normais de operação". Um requisito mensurável permite definir um teste de aceitação.

Documento de Requisitos

É a declaração oficial daquilo que é demandado dos desenvolvedores. Deve apresentar necessidades dos usuários, especificações do sistema, restrições e critérios de aceitação. Não é documento de projeto: na etapa de requisitos, descrevemos O QUE o sistema deve fazer; o COMO será implementado pertence ao projeto/design.

Quatro atividades da Engenharia de Requisitos



As quatro atividades genéricas apresentadas são: estudo de viabilidade, obtenção e análise, especificação e validação. Elas são interdependentes e iterativas: mudanças podem obrigar a revisar etapas anteriores.

O estudo de viabilidade avalia alinhamento com os objetivos da organização, viabilidade técnica, orçamento/prazo e integração com sistemas existentes.

Técnicas de levantamento

Técnica	Quando ajuda	Cuidado / limitação
Entrevista	Obter detalhes com pessoas que conhecem o processo.	Fazer perguntas abertas, neutras e registrar respostas; evitar induzir a resposta.
Revisão de documentação	Analisar manuais, relatórios, registros, diagramas e sistemas existentes.	Documentos podem estar desatualizados; combinar com outras técnicas.
Questionário	Ouvir muitos usuários, inclusive dispersos geograficamente.	Pode ter baixa participação e pouca interação para aprofundar respostas.
Observação	Ver como o trabalho ocorre no ambiente real.	Pedir autorização, não atrapalhar e registrar sistematicamente.
Brainstorm	Gerar ideias em grupo sem críticas imediatas.	Registrar todas as ideias; selecionar depois.
JAD	Sessões estruturadas com vários interessados e papéis definidos.	Exige coordenação e preparação; é mais formal que brainstorm.

Pergunta de entrevista: melhor e pior

Pior: “O sistema atual é lento e difícil, não é?” (induz a resposta). Melhor: “Como você avalia o desempenho do sistema no seu dia a dia? O que funciona bem e o que precisa melhorar?” (permite uma resposta livre).

Base no material do professor: PDF enviado, pp. 158–165 e 240–275.

4. Modelagem e UML: o começo dos diagramas

O que é modelagem?

Modelar é representar seletivamente a realidade. Um modelo não mostra tudo: destaca o que importa para uma finalidade. Abstração é simplificar, mantendo o essencial. Um diagrama é uma visão gráfica de parte de um modelo.

Analogia: mapas

Um mapa de metrô não mostra cada prédio, árvore ou distância exata da cidade. Ele mostra linhas e conexões para ajudar você a chegar ao destino. O mapa não está errado por omitir detalhes; ele foi feito para responder a uma pergunta específica. Cada diagrama UML funciona de forma parecida.

UML não é metodologia

UML significa Unified Modeling Language (Linguagem de Modelagem Unificada). É uma linguagem visual padronizada para representar e comunicar aspectos de um sistema. Não determina a ordem das etapas do projeto. RUP e Scrum organizam o processo; UML fornece notação para representar modelos. Um projeto pode usar uma metodologia com ou sem UML.

Modelo x diagrama x processo x ferramenta

Termo	O que significa	Analogia
Requisito	Necessidade ou restrição do sistema.	O que o cliente pediu.
Modelo	Representação estruturada do sistema.	A planta completa.
Diagrama	Visão gráfica de uma parte do modelo.	Uma planta elétrica ou hidráulica.
Processo	Sequência de atividades de desenvolvimento.	As etapas da obra.
Ferramenta	Software que apoia a modelagem.	O programa de desenho usado pelo arquiteto.

Níveis de abstração

1. Conceitual: entender o problema e o vocabulário do negócio.
2. Análise: representar classes do domínio, regras e comportamentos.
3. Projeto: acrescentar decisões técnicas e padrões.
4. Implementação / infraestrutura: aproximar o modelo de código, banco de dados, servidores e implantação.

Os 14 diagramas UML 2.5.1

O material apresenta o panorama dos 14 diagramas. Uma maneira útil de agrupá-los é estrutural (como o sistema é organizado) e comportamental (o que acontece e como os elementos interagem). Os quatro diagramas de interação são sequência, comunicação, visão geral de interação e tempo.

Grupo	Diagrama	Pergunta que responde
Estrutural	Classes	Quais conceitos, atributos, operações e relacionamentos existem?
Estrutural	Objetos	Como estão as instâncias concretas em um momento?
Estrutural	Pacotes	Como os elementos são agrupados em áreas/módulos?
Estrutural	Componentes	Quais módulos de software oferecem/precisam de interfaces?
Estrutural	Estrutura composta	Como um componente/classe se organiza internamente?
Estrutural	Implantação	Onde o software e seus artefatos são executados?
Estrutural	Perfis	Como a linguagem UML é estendida para um domínio específico?
Comportamental	Casos de uso	Quem usa o sistema e quais objetivos deseja alcançar?
Comportamental	Atividades	Qual é o fluxo do processo, com decisões e paralelismo?
Comportamental	Máquina de estados	Qual é o ciclo de vida de um elemento?
Interação	Sequência	Qual é a ordem temporal das mensagens?
Interação	Comunicação	Quais participantes se conectam e trocam mensagens?
Interação	Visão geral de interação	Como várias interações se encadeiam em alto nível?
Interação	Tempo / temporização	Como estados ou valores mudam ao longo de uma escala temporal?

Importante

Você não precisa desenhar os 14 em todo projeto. Escolha o diagrama pela pergunta, pelas decisões e pelos riscos do sistema. O material do professor diz explicitamente que diagramas sem finalidade geram trabalho sem valor.

Base no material do professor: PDF enviado, pp. 280–284 e 328–350.

5. Diagramas principais: aprenda a construir

5.1 Diagrama de Casos de Uso

Mostra os objetivos que atores externos alcançam usando o sistema. É muito usado para representar requisitos funcionais do ponto de vista do usuário. Não desenha telas, botões ou detalhes técnicos internos.

Analogia: cardápio de um restaurante

O cliente não quer saber como o cozinheiro corta os ingredientes. Ele quer saber o que pode pedir: consultar cardápio, fazer pedido, pagar, acompanhar entrega. Os casos de uso são os objetivos disponíveis para quem está fora do sistema.

Elementos

Elemento	Como aparece	Significado
Ator	Boneco ou nome do papel fora do limite.	Pessoa, sistema externo ou evento/tempo que interage com o sistema.
Caso de uso	Elipse com verbo + objetivo.	Resultado de valor oferecido pelo sistema.
Limite do sistema	Retângulo envolvendo os casos de uso.	Define o que pertence ao sistema.
Associação	Linha entre ator e caso de uso.	Indica participação/interação.
«include»	Seta tracejada do caso base para o caso incluído.	Comportamento obrigatório, sempre executado.
«extend»	Seta tracejada da extensão para o caso base.	Comportamento opcional/condicional; o caso base continua completo sozinho.
Generalização	Linha com triângulo vazio apontando ao elemento geral.	Especialização de ator ou caso de uso.

Como criar

1. Defina o limite: qual sistema está sendo desenhado?
2. Identifique atores externos: quem ou o que interage com ele?
3. Identifique objetivos com valor para o ator. Nomeie com verbo + objetivo: “Solicitar empréstimo”, não “Tela de empréstimo”.
4. Ligue os atores aos casos de uso e só use include/extend quando fizer sentido.
5. Descreva cada caso importante por escrito, incluindo fluxo principal, alternativas, exceções, pré-condições e pós-condições.

Exemplo de leitura: dentro do limite “Sistema de empréstimos” ficam os casos “Consultar equipamentos”, “Solicitar empréstimo”, “Registrar retirada” e “Registrar devolução”. Aluno se liga aos dois primeiros; Funcionário se liga aos dois últimos.

Memorize a diferença

INCLUDE = sempre acontece; é uma parte obrigatória reutilizada. EXTEND = só acontece em uma condição; o caso base funciona sem ela. Include: seta base → incluído. Extend: seta extensão → base.

Exemplo textual de caso de uso

Campo	Exemplo: Solicitar empréstimo
Objetivo	Permitir que aluno ou professor solicite um equipamento disponível.
Ator principal	Aluno ou professor.
Gatilho	Solicitante escolhe um equipamento e pede o empréstimo.
Pré-condições	Solicitante identificado; equipamento cadastrado e disponível.
Fluxo principal	Selecionar equipamento → sistema verifica disponibilidade → solicitante confirma → sistema registra empréstimo.
Alternativa	Se o equipamento estiver indisponível, o sistema informa e não cria empréstimo ativo.
Pós-condições	Empréstimo registrado; equipamento passa a Emprestado; retirada fica rastreável.

Base no material do professor: PDF enviado, pp. 286–297 e 289–293 para a notação; atividade integradora, pp. 339–344.

5.2 Diagrama de Atividades

Representa o fluxo de ações de um processo. Mostra a ordem das atividades, alternativas, decisões e ações paralelas. Pode usar partições (raias) para indicar quem é responsável por cada ação.

Analogia: receita de bolo

A receita mostra “misturar”, “assar”, “esperar esfriar” e decisões como “está assado?”. O diagrama de atividades representa o caminho do processo. As raias são como colunas mostrando quem faz cada etapa: aluno, funcionário ou sistema.

Elemento	Significado
Nó inicial (●)	Onde o fluxo começa.
Ação (retângulo arredondado)	Uma unidade de trabalho.
Decisão (losango)	Escolhe um caminho conforme condição/guarda.
Merge (losango)	Reúne caminhos alternativos.
Fork (barra grossa)	Divide o fluxo em atividades paralelas.
Join (barra grossa)	Sincroniza fluxos paralelos.
Partição / raia	Agrupa ações pelo responsável.

Elemento	Significado
Nó final (alvo)	Encerra a atividade/fluxo.

```
● Início
  ↓
[Solicitante escolhe equipamento]
  ↓
[Sistema consulta disponibilidade]
  ↓
◇ Está disponível?
  ├── [Sim] → [Registrar empréstimo] → [Confirmar ao solicitante] → ◎ Fim
  └── [Não] → [Informar indisponibilidade] → ◎ Fim
```

Pegadinha: decisão x fork

Decisão = escolhe apenas um caminho (por exemplo, SIM ou NÃO). Fork = abre caminhos paralelos, que acontecem em paralelo; Join = espera/reúne os fluxos paralelos. Não confunda “escolher um caminho” com “executar vários ao mesmo tempo”.

Base no material do professor: PDF enviado, pp. 298–300 e roteiro de modelagem, pp. 337–342.

5.3 Diagrama de Classes

Representa a estrutura do domínio: classes, atributos, operações e os relacionamentos entre elas. É como uma planta de organização dos conceitos do sistema.

Analogia: molde e ficha

A classe é um molde de ficha: define que toda pessoa tem nome e telefone. O objeto é uma ficha específica preenchida: “Carlos, telefone (11) 90000-0000”. A classe diz o que pode existir; o objeto mostra uma instância concreta.

Formato da classe

```
+-----------------------------------+
|            Cliente                | ← nome da classe
+-----------------------------------+
| - nome: String                    | ← atributos
| - telefone: String                |
+-----------------------------------+
| + atualizarTelefone(): void       | ← operação/método
+-----------------------------------+
```

Símbolo	Visibilidade	Leitura
+	Pública	Pode ser acessado externamente conforme o modelo.
-	Privada	Acesso restrito à própria classe.
#	Protegida	Acesso pela classe e subclasses, conforme a linguagem/modelo.
~	De pacote	Acesso dentro do pacote.

Multiplicidade

Notação	Significado
1	Exatamente um.
0..1	Zero ou um (opcional).
* ou 0..*	Zero ou muitos.
1..*	Um ou muitos (pelo menos um).
2..5	Entre dois e cinco.

Como ler: a multiplicidade colocada ao lado de uma classe informa quantas instâncias daquela classe podem estar relacionadas a uma instância da classe oposta. Exemplo: um Aluno pode ter 0..* Empréstimos; cada Empréstimo pertence a exatamente 1 Aluno.

Relacionamentos

Relacionamento	Notação / ideia	Exemplo e interpretação
Associação	Linha simples. Vínculo estrutural.	Aluno — Empréstimo.
Dependência	Seta tracejada. Uso temporário.	Serviço usa Validador.
Generalização	Triângulo vazio. Herança / “é um”.	Professor é uma Pessoa.
Realização	Triângulo vazio + linha tracejada. Implementação de interface.	EmailNotificador implementa INotificador.
Composição	Losango preenchido no lado do todo; relação forte.	Pedido ◆— ItemPedido; o item pertence ao pedido.
Agregação	Losango vazio no lado do todo; relação mais fraca.	Turma ◇— Disciplina; não implica o mesmo vínculo forte da composição.

Cuidados que o professor destaca

Não use herança sem uma relação “é-um” real. Não transforme automaticamente classe em tabela de banco de dados. Composição pressupõe propriedade forte. E multiplicidade não resolve concorrência: duas pessoas tentando ocupar a última vaga exigem controle na implementação.

Base no material do professor: PDF enviado, pp. 302–311.

5.4 Diagrama de Objetos

Mostra instâncias concretas e seus valores em um momento específico. É um “instantâneo” da estrutura em funcionamento e pode ajudar a validar se o diagrama de classes permite criar exemplos coerentes.

```
ana:Aluno
nome = "Ana Lima"

esManha:Turma
id = "T-ES-2026-1"
```

emprestimo01:Emprestimo
status = "Ativo"

Vínculos: emprestimo01 → ana; emprestimo01 → notebook01

Diferença que cai em prova

Diagrama de classes = molde geral. Diagrama de objetos = exemplo preenchido. Classe “Aluno”; objeto “ana:Aluno”.

Base no material do professor: PDF enviado, p. 310.

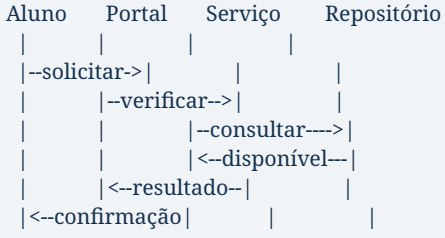
5.5 Diagrama de Sequência

Detalha uma interação e a ordem temporal das mensagens entre atores, objetos ou componentes. A leitura é de cima para baixo. As linhas verticais tracejadas são linhas de vida; ativações mostram períodos de execução.

Analogia: conversa no WhatsApp

Você manda uma mensagem, o aplicativo encaminha ao servidor, o servidor pergunta ao banco de dados e recebe uma resposta. O diagrama registra quem falou com quem e em qual ordem. Diferente do diagrama de atividades, o foco não é somente o fluxo, mas os participantes e as mensagens entre eles.

Elemento	Como interpretar
Participante / lifeline	Ator, objeto ou componente que participa.
Linha de vida	Linha vertical tracejada; existência do participante no cenário.
Ativação	Retângulo sobre a linha de vida; execução de comportamento.
Mensagem síncrona	Chamada em que o emissor aguarda o término/resposta.
Mensagem assíncrona	Envio em que o emissor pode prosseguir sem aguardar.
Retorno	Resposta, geralmente indicada por linha tracejada.
alt	Alternativas condicionais, como aprovado ou recusado.
opt	Trecho opcional, como aplicar cupom.
loop	Repetição, como processar cada item.
par	Execuções concorrentes/paralelas.



```
alt [disponível] → registrar empréstimo e confirmar
else [indisponível] → informar que não há equipamento
```

Atenção

A distância vertical entre mensagens não indica automaticamente duração. Desenhar uma chamada como atômica também não garante que a implementação tenha controle de concorrência.

Base no material do professor: PDF enviado, pp. 312–314 e 398–405.

5.6 Diagrama de Máquina de Estados

Mostra o ciclo de vida de um elemento específico: em quais estados pode estar e quais eventos provocam a mudança entre eles. Não descreve o fluxo completo de trabalho da equipe; descreve a situação de um objeto ao longo do tempo.

Analogia: rastrear um pedido

Um pedido pode estar Aberto, Pago, Em Entrega, Entregue ou Cancelado. “Pagar” é um evento que pode levar de Aberto para Pago. O estado responde “em que situação o pedido está agora?”.

Elemento	Significado
Estado inicial (●)	Ponto de partida.
Estado	Situação estável do objeto, como Disponível ou Emprestado.
Transição (seta)	Mudança de estado.
Evento	Ocorrência que dispara a transição.
Guarda [condição]	Condição que precisa ser verdadeira para permitir a transição.
Efeito / ação	Comportamento executado durante a transição.
entry / exit / do	Comportamentos ao entrar, sair ou durante um estado.
Estado final	Indica o encerramento do ciclo modelado.

```
● → Disponível --reservar--> Reservado
Reservado --retirar--> Emprestado
Emprestado --devolver em bom estado--> Disponível
Emprestado --devolver danificado--> Em Manutenção
Em Manutenção --liberar após reparo--> Disponível
```

Sintaxe comum de transição: evento [guarda] / efeito. Exemplo: confirmar [vaga disponível] / registrarAuditoria().

Atividades x estados

Atividades perguntam “o que acontece no processo?”. Estados perguntam “em que situação o objeto está?”. Um processo de devolver equipamento pode ter várias ações; o equipamento, por sua vez, pode passar de Emprestado para Disponível ou Em Manutenção.

Base no material do professor: PDF enviado, pp. 316–318 e exemplos de empréstimo, p. 342.

6. Os outros diagramas UML

Além dos seis diagramas aprofundados acima, memorize a função dos outros oito. A prova pode apresentar uma descrição e perguntar qual desenho é adequado.

Diagrama	O que mostra	Analogia / quando escolher
Comunicação	Participantes, conexões e mensagens; usa numeração hierárquica para indicar ordem/chamadas.	Mapa de quem está conectado a quem; use quando o foco são vínculos e responsabilidades, não o eixo temporal.
Visão Geral de Interação	Fluxo de alto nível que organiza várias interações, podendo referenciar diagramas de sequência.	Mapa de capítulos de uma conversa longa; mostra como cenários se encadeiam.
Tempo / Temporização	Mudança de estados ou valores ao longo de uma escala de tempo explícita.	Linha do tempo de uma reserva que expira após um prazo.
Pacotes	Agrupa elementos relacionados e mostra dependências entre grupos.	Pastas de um computador: Acadêmico, Identidade, Comunicação.
Componentes	Módulos de software, interfaces oferecidas/requeridas e dependências.	Peças de LEGO com encaixes/contratos; mostra quem depende de quem.
Estrutura Composta	Partes, portas e conectores que revelam a organização interna de um componente/classe.	Abrir um aparelho para ver os módulos internos.
Implantação	Nós, dispositivos, ambientes de execução, artefatos e caminhos de comunicação.	Mapa físico de onde os programas rodam: celular, servidor web, aplicação e banco.
Perfis	Extensões da UML por meio de estereótipos, propriedades e restrições definidas para um domínio.	Criar um conjunto de etiquetas e regras específicas para modelar um tipo de sistema.

Diferenças que não podem se misturar

Não confunda	Diferença essencial
Componentes x implantação	Componentes = organização lógica e contratos de software. Implantação = onde e como os artefatos rodam fisicamente.
Atividades x sequência	Atividades = fluxo de ações e decisões. Sequência = mensagens entre participantes em ordem temporal.
Atividades x estados	Atividades = processo. Estados = ciclo de vida de um objeto.
Classes x objetos	Classes = estrutura geral. Objetos = instâncias concretas com valores.
Comunicação x sequência	Ambos representam interação. Comunicação enfatiza conexões; sequência enfatiza a ordem temporal.
Pacotes x componentes	Pacotes agrupam elementos do modelo; componentes são unidades modulares de software com interfaces e dependências.

Base no material do professor: PDF enviado, pp. 315 e 319–328.

7. Arquitetura, consistência e rastreabilidade

MVC: Model, View, Controller

Parte	Responsabilidade	Analogia
Model	Dados, regras e comportamento da aplicação.	A cozinha e as regras do restaurante.
View	Apresentação visual e interface.	O cardápio e o que o cliente vê.
Controller	Recebe interações do usuário e coordena View e Model.	O atendente que recebe o pedido e encaminha à cozinha.

SOA e arquitetura baseada em componentes

SOA (Service-Oriented Architecture) disponibiliza funcionalidades como serviços que podem ser consumidos por outras aplicações. O ESB (Enterprise Service Bus) intermedeia a comunicação, o roteamento e a transformação de mensagens. Na arquitetura baseada em componentes, o sistema é dividido em módulos com responsabilidades definidas e interfaces padronizadas.

Consistência entre diagramas

Todos os diagramas do mesmo projeto precisam contar a mesma história. Se “Cancelar matrícula” aparece nos casos de uso, o estado Cancelada precisa aparecer na máquina de estados e o comportamento deve estar coerente na descrição textual, nas regras e nos testes. Não adianta cada diagrama estar bonito se os desenhos se contradizem.

Rastreabilidade: requisito → modelo → teste

Requisito / regra	Modelo que ajuda a representar	Exemplo de teste
Solicitar matrícula válida	Caso de uso + sequência	Confirmar solicitação válida gera matrícula e comprovante.
Pré-requisito pendente	Fluxo alternativo do caso de uso	Recusar aluno sem pré-requisito concluído.
Capacidade máxima da turma	Restrição + sequência	Impedir matrícula quando não há vagas.
Não permitir duplicidade	Regra + sequência	Recusar segunda matrícula ativa na mesma turma.
Cancelamento dentro do prazo	Máquina de estados	Aceitar dentro do prazo; recusar fora dele.

Lembrete

Diagramas ajudam a descobrir problemas, discutir soluções e planejar testes, mas não provam que o sistema está correto. A correção precisa ser verificada com validações, testes e execução.

Base no material do professor: PDF enviado, pp. 166–170, 237–239 e 329–348.

8. Receita para resolver questões práticas de modelagem

Antes de desenhar

1. Leia o cenário inteiro. Descubra o objetivo do sistema e o que está fora do limite dele.
2. Marque substantivos: aluno, equipamento, empréstimo, pagamento, pedido. Eles podem virar classes.
3. Marque verbos: consultar, solicitar, confirmar, cancelar, registrar. Eles podem virar casos de uso ou ações.
4. Marque condições: disponível, vencido, aprovado, recusado, danificado. Elas podem virar guardas, alternativas ou estados.
5. Anote regras de negócio: limites, prazos, proibições e exceções.
6. Escolha um diagrama por vez. Não misture no mesmo desenho as perguntas de todos os diagramas.
7. No final, compare os diagramas: nomes, regras, atores, estados e mensagens precisam combinar.

Estudo de caso integrado: empréstimo de notebook

O cenário didático do material do professor: alunos e professores solicitam notebooks e projetores; funcionários registram retirada e devolução; equipamento em manutenção não pode ser emprestado; não pode haver dois empréstimos ativos para o mesmo equipamento; equipamento danificado pode ir para manutenção; reserva pode ser cancelada antes da retirada.

Atores e objetivos

Atores possíveis: Aluno, Professor e Funcionário. Objetivos: consultar equipamentos, solicitar empréstimo, reservar equipamento, cancelar reserva, registrar retirada, registrar devolução e encaminhar equipamento para manutenção.

Classes candidatas

Solicitante, Equipamento, Empréstimo e Reserva. O objeto Empréstimo liga um solicitante a um equipamento. A Reserva representa intenção futura; Empréstimo representa vínculo ativo. Não trate conceitos diferentes como sinônimos.

Estados do equipamento

```
Disponível → Reservado → Emprestado → Disponível
                    ↘ Em Manutenção → Disponível
Reserva cancelada antes da retirada: volta a Disponível (conforme regra do domínio).
```

Sequência de solicitação

```
Solicitante → Portal: solicitar(equipamento)
Portal → Serviço: verificarDisponibilidade()
Serviço → Repositório: consultarStatus()
Repositório → Serviço: status
alt [disponível] → registrarEmpréstimo() → confirmar
else [indisponível/manutenção] → informar indisponibilidade
```

Testes que nascem das regras

- Tentar emprestar equipamento disponível: a solicitação deve ser registrada.
- Tentar emprestar equipamento em manutenção: o sistema deve recusar.
- Tentar criar dois empréstimos ativos para o mesmo equipamento: somente um pode ser confirmado.
- Devolver equipamento danificado: ele deve ser encaminhado para manutenção.
- Cancelar reserva antes da retirada: o cancelamento deve ser registrado e a disponibilidade atualizada.

9. Métodos ágeis: Scrum, Kanban e outras abordagens

O material final da disciplina apresenta agilidade como a capacidade de responder a mudanças e aprender durante o desenvolvimento, sem perder o foco no valor entregue ao usuário. Isso não significa trabalhar sem planejamento: significa planejar, observar o resultado, receber feedback e adaptar o caminho.

Analogia: organizar uma cantina

Imagine que a equipe está criando um aplicativo para a cantina. No começo, pensava em colocar pagamento online primeiro. Depois de conversar com quem atende, percebe que o maior problema é a fila na retirada. Uma equipe ágil não ignora o planejamento: avalia a nova necessidade, conversa com o cliente, reorganiza prioridades e entrega uma melhoria útil mais cedo.

Incremental x iterativo

Abordagem	O que significa	Exemplo na cantina	Analogia
Incremental	Acrescentar partes utilizáveis ao produto.	Primeiro consultar cardápio; depois fazer pedidos; depois acompanhar a retirada.	Construir uma biblioteca por seções, que já podem ser usadas.
Iterativa	Revisar e aperfeiçoar algo com base no aprendizado.	Melhorar a tela do cardápio depois que alunos relatam dificuldades.	Testar uma receita e ajustar o tempero a cada tentativa.
Combinadas	Ampliar o produto e melhorar o que já existe.	Entregar pedidos e, ao mesmo tempo, aperfeiçoar o cardápio.	Adicionar um cômodo e melhorar a planta com o que se aprendeu.

Macete

Incremental = acrescentar. Iterativo = repetir para melhorar. Um projeto pode ser os dois ao mesmo tempo.

Manifesto Ágil: os quatro valores

O Manifesto Ágil foi publicado em 2001. Ele não diz que os itens da direita não têm valor; diz que, diante de escolhas, os itens da esquerda recebem maior prioridade:

- Indivíduos e interações mais que processos e ferramentas: conversa e colaboração resolvem dúvidas que uma ferramenta sozinha não resolve.
- Software em funcionamento mais que documentação abrangente: uma versão utilizável permite verificar se o produto resolve o problema. Documentação continua importante.

- Colaboração com o cliente mais que negociação de contratos: cliente e equipe conversam ao longo do desenvolvimento. Contratos continuam importantes.
- Responder a mudanças mais que seguir um plano: o plano orienta o trabalho, mas pode ser revisto quando novas informações surgem.

Princípios ágeis em linguagem simples

1. Entregar valor cedo e continuamente, em vez de deixar o usuário esperando até o fim.
2. Aceitar mudanças nos requisitos e avaliar seus impactos, mesmo durante o projeto.
3. Entregar software funcionando com frequência.
4. Manter colaboração diária entre pessoas do negócio e desenvolvedores.
5. Dar apoio e confiança a pessoas motivadas.
6. Preferir comunicação direta e eficaz.
7. Usar software funcionando como principal medida de progresso.
8. Manter ritmo sustentável, sem depender de esforço excessivo constante.
9. Valorizar excelência técnica e bom projeto para facilitar mudanças.
10. Buscar simplicidade e reduzir trabalho desnecessário.
11. Permitir que equipes organizem seu próprio trabalho.
12. Refletir periodicamente e ajustar a forma de trabalhar.

Scrum: pilares, valores e responsabilidades

Scrum é um framework leve para gerar valor por meio de soluções adaptativas para problemas complexos. No material, seus três pilares são transparência, inspeção e adaptação: deixar o trabalho visível, examinar o que está acontecendo e ajustar o caminho com base no que foi aprendido.

Responsabilidade	Papel principal	Analogia
Product Owner	Maximiza o valor do produto e gerencia o Product Backlog, a lista ordenada das necessidades.	Pessoa que organiza o que a cantina mais precisa melhorar.
Scrum Master	Ajuda a estabelecer o Scrum e melhora a efetividade da equipe; não atua como chefe que distribui tarefas.	Facilitador que remove obstáculos e ajuda o grupo a trabalhar melhor.
Developers	Profissionais que criam incrementos utilizáveis e cuidam da qualidade.	Equipe que planeja, constrói, testa e integra a solução.

Valores do Scrum: compromisso, foco, abertura, respeito e coragem. São comportamentos que sustentam a colaboração e o trabalho em equipe.

Eventos do Scrum

Evento	Para que serve	Lembrete
Sprint	Período de duração fixa de até um mês; contém os demais eventos.	Duas semanas é um exemplo possível, não uma regra fixa.
Sprint Planning	Planejar por que a Sprint tem valor, o que será feito e como realizar o trabalho.	Planejamento da Sprint.

Evento	Para que serve	Lembrete
Daily Scrum	Evento de 15 minutos para Developers inspecionarem o progresso e adaptarem o plano.	Não é prestação de contas ao chefe; as três perguntas tradicionais não são obrigatórias.
Sprint Review	Inspecionar o resultado com stakeholders e discutir adaptações futuras.	Olha para o produto.
Sprint Retrospective	Refletir sobre a forma de trabalhar e planejar melhorias.	Olha para o processo/equipe.

Review x Retrospective

Review = revisar o produto com as partes interessadas. Retrospective = revisar como a equipe trabalhou e o que pode melhorar.

Artefatos e compromissos do Scrum

Artefato	O que representa	Compromisso associado
Product Backlog	Lista evolutiva e ordenada de necessidades e melhorias do produto.	Objetivo do Produto.
Sprint Backlog	Objetivo da Sprint, itens selecionados e plano para realizá-los.	Objetivo da Sprint.
Incremento	Resultado utilizável e integrado ao que já foi produzido.	Definição de Pronto.

A Definição de Pronto é o entendimento compartilhado das condições de qualidade que o incremento precisa cumprir. Exemplo: código revisado, testes executados e integração verificada.

Kanban: visualizar e melhorar o fluxo

Kanban torna o trabalho visível por meio de um quadro com etapas, como “A fazer”, “Em desenvolvimento”, “Em teste” e “Concluído”. Cada cartão representa um item de trabalho. A equipe explicita regras, acompanha gargalos e limita o WIP (Work in Progress), isto é, a quantidade de trabalho iniciado e ainda não concluído.

Analogia: cozinha com pedidos demais

Se a cozinha começa dez pedidos ao mesmo tempo, mas só consegue terminar dois, todos ficam esperando. Limitar o WIP ajuda a equipe a concluir o que já começou antes de puxar mais trabalho.

Atenção: um quadro sozinho não é todo o Kanban. Kanban não exige Sprints e pode apoiar um fluxo contínuo. Scrum organiza o trabalho em Sprints e eventos; Kanban concentra a atenção no fluxo e nos limites de trabalho em andamento.

XP, TDD, RAD e FDD

Abordagem / prática	Ênfase	Exemplo / conceito para lembrar
XP (Extreme Programming)	Práticas técnicas e feedback frequente.	Programação em pares e integração contínua.

Abordagem / prática	Ênfase	Exemplo / conceito para lembrar
TDD (Test-Driven Development)	Ciclo curto: teste que falha → implementação suficiente → refatoração.	Escrever primeiro o teste do cálculo do pedido; depois implementar para fazê-lo passar.
Refatoração	Melhorar a estrutura interna sem alterar o comportamento observável.	Extrair uma função repetida sem mudar o resultado do programa.
RAD (Rapid Application Development)	Prototipação, participação dos usuários e ciclos de feedback.	Protótipo navegável para entender como destacar produtos indisponíveis.
FDD (Feature-Driven Development)	Modelar o domínio e desenvolver por pequenas funcionalidades com valor.	"Calcular o total do pedido" é uma funcionalidade; "criar uma tabela" é tarefa técnica.

TDD: 1) escrever um teste que falha → 2) implementar o mínimo → 3) refatorar
REFATORAR: melhorar a estrutura sem mudar o comportamento.
KANBAN: visualizar o fluxo + limitar WIP + identificar gargalos.
SCRUM: Sprint + planejamento + Daily + Review + Retrospective.

Como escolher e combinar abordagens

Scrum organiza objetivos, responsabilidades e eventos. Kanban melhora a visualização e o fluxo. XP reforça práticas técnicas. RAD usa protótipos e participação dos usuários. FDD organiza a construção por funcionalidades. Uma equipe pode combinar práticas quando isso fizer sentido: por exemplo, Scrum para organizar o trabalho, Kanban para enxergar o fluxo e TDD para desenvolver com qualidade.

Agilidade não elimina requisitos, UML ou documentação

Requisitos e diagramas continuam úteis; eles evoluem quando a equipe aprende algo novo. Uma mudança na retirada da cantina, por exemplo, pode exigir revisar requisitos, caso de uso, classes, atividades, sequência e testes. Documentação útil e atualizada ajuda a equipe a comunicar decisões e manter o produto.

Base no material do professor: PDF enviado, pp. 415–460. Os conceitos de Scrum seguem a abordagem apresentada no material, que referencia o Scrum Guide de 2020.

10. Vídeos em português para estudar

Sugestão: assista na ordem indicada. Comece pela visão geral; depois escolha aulas específicas conforme a dificuldade. Os vídeos abaixo foram localizados em fontes públicas. Alguns são aprofundamentos, então não precisa começar pelos conteúdos avançados.

1. DevMedia — Curso de UML: aula demonstrativa — https://www.youtube.com/watch?v=2dSq0Vu1GFo Boa introdução para entender o que é UML e para que servem os diagramas.

2. Prof. Gilleanes Guedes — Diagrama de Casos de Uso, Parte I — https://www.youtube.com/watch?v=qVemjrDKw2k Atores, casos de uso e documentação; combina bem com o conteúdo do professor.

3. Gabriel Skrilatt — Modelagem UML: caso de uso, classe e sequência — https://www.youtube.com/watch?v=pCCqiJDLr4Y Exemplo prático com ferramenta de modelagem.

4. DevMedia — Curso de Diagrama de Classes — https://www.devmedia.com.br/curso/introducao-a-construcao-de-diagrama-de-classes-da-uml/307 Série com atributos, operações, associação, herança, agregação, composição e construção de exemplo.

5. e-Aulas USP — Diagrama de Classes — https://eaulas.usp.br/portal/video?idItem=27545 Aula universitária sobre classes e notação UML.

6. Prof. Gilleanes Guedes — Diagrama de Atividades, Parte III — https://www.youtube.com/watch?v=HBJvuhkX-Vs Aprofunda partições/raias, bifurcação, união e exemplos. É uma aula avançada.

7. Eduardo Engenharia de Software — Diagrama de Sequência — https://www.youtube.com/watch?v=6khudydsYYQ Atores, objetos, linhas de vida e ativações.

8. Prof. Gilleanes Guedes — Diagrama de Sequência, exemplos — https://www.youtube.com/watch?v=Xzatg5Fz2eQ Exemplos práticos de processos e interações.

9. Fabio de Lima Bezerra — Linguagem UML: Máquina de Estados — https://www.youtube.com/watch?v=N5gfaNcdYjI Estado inicial/final, estados e transições.

10. Scrum.org — Scrum em 21 minutos (em português) — https://www.scrum.org/resources/blog/scrum-em-21-minutos Vídeo curto apresentado por trainer brasileiro; visão geral de papéis, eventos e artefatos.

11. Celso Kitamura — O que é Kanban? — https://www.youtube.com/watch?v=MuIW3__Srh0 Explica quadro, cartões, colunas e fluxo de trabalho.

12. Economia para Iniciantes — O que é Kanban na prática? — https://www.youtube.com/watch?v=CqA5XKlraOk Exemplos de colunas, gargalos e limite WIP.

13. Coursera — TDD: Desenvolvimento de Software Guiado por Testes — https://www.coursera.org/learn/tdd-desenvolvimento-de-software-guiado-por-testes Curso mais aprofundado; pode exigir inscrição e é opcional para esta revisão.

Como assistir de forma ativa

Pause o vídeo antes do exemplo resolvido e tente desenhar sozinha. Depois compare: identificou os atores? colocou o limite do sistema? marcou as guardas? leu as multiplicidades pelo lado correto? Anote apenas o erro que você cometeu e refaça o desenho.

11. Revisão rápida: o que decorar

Se aparecer no enunciado...	Pense em...
“Usuário”, “objetivo”, “funcionalidade”, “ator”	Caso de uso
“Etapas”, “fluxo”, “decisão”, “sim/não”, “raias”	Atividades
“Atributo”, “método”, “relacionamento”, “multiplicidade”	Classes
“Instância”, “exemplo concreto”, “valores preenchidos”	Objetos
“Mensagem”, “ordem”, “participantes”, “resposta”	Sequência
“Situação”, “ciclo de vida”, “mudança de status”, “evento”	Máquina de estados
“Conexões entre objetos”, “numeração de mensagens”	Comunicação
“Agrupar módulos”, “dependência entre áreas”	Pacotes
“Interface fornecida/requerida”, “módulo de software”	Componentes
“Hardware”, “servidor”, “nó”, “artefato implantado”	Implantação
“Prazo”, “timeout”, “mudança em escala temporal”	Tempo

Se aparecer no enunciado...	Pense em...
“Organização interna de partes e conectores”	Estrutura composta

Notações-relâmpago

CASOS DE USO: ator + elipse + limite do sistema
INCLUDE: sempre; seta do caso base para o incluído
EXTEND: opcional; seta da extensão para o caso base
CLASSE: nome | atributos | operações
VISIBILIDADE: + público | - privado | # protegido | ~ pacote
MULTIPLICIDADE: 1 | 0..1 | 0..* | 1..* | 2..5
ATIVIDADE: ● início | retângulo ação | losango decisão | barras fork/join | final
SEQUÊNCIA: participantes no topo; linhas de vida; mensagens de cima para baixo
ESTADOS: estado + evento [guarda] / efeito

Três erros para evitar

1) Desenhar telas em vez de objetivos no caso de uso. 2) Confundir decisão (um caminho) com fork (paralelismo). 3) Misturar o fluxo de atividades com o ciclo de vida do objeto. Se evitar esses três, já elimina muitas confusões comuns.

12. Lista de exercícios — 50 questões

Faça sem consultar os resumos. Nas questões de desenho, não precisa usar uma ferramenta: lápis e papel são suficientes, desde que os símbolos, nomes, setas e condições estejam claros.

Parte A — Múltipla escolha (1 a 20)

1. Qual é o principal objetivo da Engenharia de Software?

- A) Escrever o maior número possível de linhas de código.
- B) Organizar o desenvolvimento e a manutenção de software com qualidade e eficiência.
- C) Escolher uma linguagem de programação para todos os projetos.
- D) Criar apenas a interface visual.

2. Qual alternativa descreve melhor o que faz parte de um software?

- A) Apenas o código-fonte.
- B) Apenas o aplicativo instalado.
- C) Código e hardware físico.
- D) Programas, dados, interfaces, documentação e materiais de apoio.

3. A visão sistêmica recomenda:

- A) Analisar cada componente isoladamente e ignorar conexões.
- B) Considerar as relações entre as partes e o impacto de mudanças no todo.
- C) Eliminar a comunicação entre equipes.
- D) Começar sempre pela codificação.

4. Qual opção contém fatores que favorecem o sucesso de um projeto?

- A) Objetivos claros, stakeholders envolvidos, planejamento e monitoramento.
- B) Ausência de cronograma e mudanças sem registro.
- C) Apenas tecnologia moderna.
- D) Evitar conversar com os usuários.

5. Qual alternativa diferencia corretamente desenvolvimento incremental e iterativo?

- A) Incremental acrescenta partes utilizáveis; iterativo revisa e aperfeiçoa a solução.
- B) Incremental elimina testes; iterativo elimina feedback.
- C) São termos que significam exatamente a mesma coisa.
- D) Iterativo só acontece depois da descontinuação.

6. O modelo cascata caracteriza-se por:

- A) Entregas parciais em ciclos com análise de riscos em cada volta.
- B) Etapas sequenciais, em que uma fase sucede a outra.
- C) Construção de protótipo sem produto final.
- D) Ausência de planejamento.

7. A prototipação é especialmente útil quando:

- A) O cliente já definiu todos os detalhes técnicos.
- B) Os requisitos estão completamente estáveis e não há dúvidas.
- C) O cliente conhece o objetivo geral, mas ainda não detalhou entradas, processamento e saídas.
- D) O sistema já foi descontinuado.

8. O modelo espiral/incremental acrescenta destaque a:

- A) Análise de riscos e desenvolvimento iterativo por incrementos.
- B) Proibição de feedback.
- C) Entrega somente ao final, sem revisões.
- D) Exclusão de protótipos.

9. Qual é a ordem correta das fases do RUP?

- A) Construção, concepção, transição, elaboração.
- B) Concepção, elaboração, construção, transição.
- C) Elaboração, transição, construção, concepção.
- D) Concepção, construção, transição, elaboração.

10. Qual frase diferencia corretamente usabilidade e UX?

- A) São sinônimos exatos.
- B) Usabilidade envolve facilidade e eficiência de uso; UX envolve percepção e resposta à experiência.
- C) UX trata apenas de velocidade do processador.
- D) Usabilidade é somente a cor da tela.

11. “O sistema deve permitir cadastrar alunos” é:

- A) Requisito funcional.
- B) Requisito não funcional.
- C) Regra de arquitetura.
- D) Um caso de uso obrigatório de implantação.

12. “O sistema deve responder às consultas em até 2 segundos” é:

- A) Requisito funcional.
- B) Requisito não funcional mensurável.
- C) Uma classe.
- D) Um ator.

13. A Engenharia de Requisitos inclui:

- A) Apenas codificação.
- B) Estudo de viabilidade, obtenção/análise, especificação e validação, em um processo iterativo.
- C) Apenas implantação física.
- D) Exclusivamente desenho de telas.

14. Qual pergunta é mais adequada em uma entrevista de levantamento?

- A) “O sistema é horrível, não é?”
- B) “Você concorda que precisamos trocar tudo?”
- C) “Como você realiza essa tarefa hoje e quais dificuldades encontra?”
- D) “A solução que pensei é a única possível, certo?”

15. O JAD diferencia-se do brainstorm porque:

- A) Não envolve usuários.
- B) É uma sessão estruturada, com papéis definidos e documentação.
- C) Proíbe registrar decisões.
- D) É apenas um questionário individual.

16. A UML é:

- A) Uma linguagem de modelagem visual.
- B) Uma linguagem de programação.
- C) Uma metodologia que obriga a usar cascata.
- D) Um banco de dados.

17. Qual é a relação correta entre modelo e diagrama?

- A) São exatamente a mesma coisa.
- B) O diagrama é uma visão gráfica de parte do modelo.
- C) O modelo é sempre um desenho de uma única página.
- D) O diagrama contém todo o código-fonte.

18. Qual diagrama é melhor para mostrar os objetivos de um usuário diante do sistema?

- A) Implantação.
- B) Casos de uso.
- C) Temporização.
- D) Estrutura composta.

19. Qual diagrama mostra atributos, operações e relações entre classes?

- A) Classes.
- B) Atividades.
- C) Sequência.
- D) Tempo.

20. Qual alternativa diferencia corretamente Scrum e Kanban?

- A) Scrum organiza o trabalho em Sprints e eventos; Kanban enfatiza o fluxo e os limites de WIP, sem exigir Sprints.
- B) Kanban exige Sprints e Scrum não possui eventos.
- C) Os dois são nomes diferentes para a mesma prática.
- D) Scrum proíbe feedback dos usuários.

Parte B — Verdadeiro ou falso (21 a 30)

21. UML define, sozinha, todas as etapas de desenvolvimento de um software. (V/F)

22. Um ator de caso de uso precisa ser sempre uma pessoa; um sistema externo nunca pode ser ator. (V/F)

23. «include» representa comportamento que sempre ocorre no caso base. (V/F)

24. «extend» representa comportamento opcional ou condicional e aponta para o caso base. (V/F)

25. No diagrama de atividades, uma decisão escolhe um caminho; fork abre caminhos paralelos. (V/F)

26. O diagrama de estados mostra as atividades de todos os participantes em uma sequência de trabalho. (V/F)

27. No diagrama de sequência, a leitura das mensagens ocorre de cima para baixo. (V/F)

28. A multiplicidade 0..* significa zero ou muitos. (V/F)

29. Composição representa uma relação forte entre o todo e suas partes. (V/F)

30. Um diagrama bem feito substitui a necessidade de validar e testar o sistema. (V/F)

Parte C — Respostas curtas (31 a 40)

31. Explique com suas palavras a diferença entre classe e objeto.

Resposta: _____

32. O que significam os símbolos +, -, # e ~ em uma classe UML?

Resposta: _____

33. Interprete a frase: “Um aluno pode possuir 0..* empréstimos; cada empréstimo pertence a 1 aluno”.

Resposta: _____

34. Diferencie associação, dependência e generalização em diagramas de classes.

Resposta: _____

35. Qual é a diferença entre diagrama de atividades e diagrama de sequência?

Resposta: _____

36. O que representa uma guarda em uma transição de máquina de estados? Dê um exemplo.

Resposta: _____

37. Para que serve um diagrama de comunicação e qual é seu foco em relação ao diagrama de sequência?

Resposta: _____

38. Diferencie diagrama de componentes e diagrama de implantação.

Resposta: _____

39. Qual é o objetivo de um diagrama de pacotes?

Resposta: _____

40. Por que requisitos não funcionais devem ser mensuráveis? Dê um exemplo.

Resposta: _____

Parte D — Aplicação e construção de diagramas (41 a 50)

41. Sistema de biblioteca: identifique pelo menos três atores e seis casos de uso. Delimite o sistema e indique uma relação «include» que faça sentido.

Rascunho / resposta:

_____

_____

42. Ainda na biblioteca, descreva em texto o caso de uso “Realizar empréstimo”, incluindo ator principal, pré-condições, fluxo principal, uma alternativa e pós-condições.

Rascunho / resposta:

_____

_____

43. Desenhe um diagrama de atividades para o empréstimo de um livro: usuário seleciona livro, sistema verifica disponibilidade; se disponível, registra empréstimo; se indisponível, informa o usuário. Use início, ações, decisão e fim.

Rascunho / resposta:

_____

_____

44. Para o sistema de empréstimo, proponha quatro classes, dois atributos por classe e os relacionamentos com multiplicidades. Explique uma das multiplicidades.

Rascunho / resposta:

_____

_____

45. Faça um diagrama de objetos que represente Ana como aluna, um livro específico e um empréstimo ativo, com valores concretos e vínculos.

Rascunho / resposta:

_____

_____

46. Faça um diagrama de sequência para realizar o empréstimo, com os participantes Usuário, Portal, Serviço de Empréstimo e Repositório. Inclua a alternativa de livro indisponível usando alt.

Rascunho / resposta:

_____

_____

47. Modele os estados de um livro: Disponível, Reservado, Emprestado e Em Manutenção. Indique eventos que disparam pelo menos três transições.

Rascunho / resposta:

_____

_____

48. Uma compra online precisa consultar estoque, autorizar pagamento e confirmar o pedido. Diga quais diagramas usaria para mostrar (a) objetivos do cliente, (b) fluxo de compra, (c) mensagens entre serviços, (d) estados do pedido. Justifique cada escolha.

Rascunho / resposta:

_____

_____

49. Desenhe uma visão simplificada de componentes e outra de implantação para um sistema web com navegador, servidor web, servidor de aplicação e banco de dados. Explique por que os desenhos não são iguais.

Rascunho / resposta:

_____

_____

50. Requisito: “não pode haver dois empréstimos ativos do mesmo livro”. Indique em quais modelos/documentos essa regra deve aparecer e proponha um teste que comprove seu atendimento.

Rascunho / resposta:

_____

_____

13. Gabarito e critérios de correção

Como usar o gabarito

Nas questões 41 a 50 existem várias soluções válidas. Compare sua resposta com os elementos obrigatórios e verifique se os desenhos contam uma história coerente. O importante não é copiar exatamente o exemplo, mas representar corretamente o cenário.

Parte A — Múltipla escolha

Questão	Resposta	Por quê
1	B	Engenharia de Software organiza desenvolvimento e manutenção com qualidade e eficiência.
2	D	Software é mais amplo que código; inclui dados, interface e documentação.
3	B	Visão sistêmica considera partes, relações e efeitos no conjunto.
4	A	Objetivos, stakeholders, planejamento e monitoramento favorecem o sucesso.
5	A	Incremental acrescenta capacidade; iterativo revisa e melhora o que já existe.
6	B	Cascata organiza fases em sequência.
7	C	Prototipação ajuda a esclarecer requisitos ainda pouco detalhados.
8	A	Espiral/incremental destaca iterações, incrementos e análise de riscos.
9	B	Concepção → Elaboração → Construção → Transição.
10	B	Usabilidade é facilidade/eficiência; UX é percepção e resposta à experiência.
11	A	Cadastrar é um serviço/ação que o sistema deve executar: RF.
12	B	Tempo de resposta é qualidade/desempenho e foi especificado com métrica.
13	B	As quatro atividades são estudo de viabilidade, obtenção/análise, especificação e validação; o processo é iterativo.
14	C	Pergunta aberta e neutra, sem induzir uma solução.
15	B	JAD é estruturado, envolve vários papéis e registra decisões.
16	A	UML é uma linguagem de modelagem, não linguagem de programação ou metodologia.
17	B	O diagrama representa graficamente uma parte/visão do modelo.
18	B	Casos de uso mostram atores e objetivos.
19	A	Classes exibem atributos, operações e relações.
20	A	Scrum trabalha com Sprints e eventos; Kanban enfatiza fluxo e limite de WIP, sem exigir Sprints.

Parte B — Verdadeiro ou falso

Questão	Gabarito	Justificativa
21	F	UML é linguagem de modelagem; metodologias/processos como RUP ou Scrum organizam o trabalho.
22	F	Ator é um papel externo; pode ser pessoa, sistema externo ou evento/tempo.

Questão	Gabarito	Justificativa
23	V	Include é comportamento sempre executado pelo caso base.
24	V	Extend é opcional/condicional; seta da extensão para o caso base.
25	V	Decisão escolhe um caminho; fork inicia execução paralela.
26	F	Máquina de estados acompanha o ciclo de vida de um elemento.
27	V	A linha temporal do diagrama de sequência é lida de cima para baixo.
28	V	0..* significa zero ou muitos.
29	V	Composição indica propriedade/dependência forte da parte em relação ao todo.
30	F	Diagramas apoiam análise; validação e testes continuam necessários.

Parte C — Respostas curtas

31. Classe é o molde que define características e operações; objeto é uma instância concreta, com valores próprios. Ex.: Cliente é classe; ana:Cliente é objeto.

32. + pública; - privada; # protegida; ~ de pacote.

33. Um aluno pode não ter nenhum empréstimo ou ter vários; cada empréstimo está associado a exatamente um aluno.

34. Associação: vínculo estrutural; dependência: uso de outra classe; generalização: herança/especialização, relação "é-um".

35. Atividades mostram o fluxo de ações/decisões do processo; sequência mostra mensagens entre participantes e sua ordem temporal.

36. Guarda é uma condição booleana que precisa ser verdadeira para a transição acontecer. Ex.: devolver [sem danos] / liberar equipamento.

37. Comunicação mostra participantes conectados e suas mensagens, com foco nas ligações; sequência enfatiza a ordem temporal.

38. Componentes mostram módulos lógicos, interfaces e dependências; implantação mostra hardware/nós, ambientes e onde os artefatos são executados.

39. WIP é o trabalho iniciado e ainda não concluído. Limitar WIP evita que a equipe comece tarefas demais, evidencia gargalos e ajuda a concluir o que já está em andamento.

40. Porque deve ser possível verificar objetivamente. Ex.: "95% das consultas respondem em até 2 segundos" ou "suportar 500 usuários simultâneos".

Parte D — Elementos esperados nas questões práticas

41. Biblioteca — atores e casos de uso

Atores possíveis: Leitor/Aluno, Bibliotecário e Serviço de Identidade. Casos de uso: Consultar catálogo, Consultar disponibilidade, Reservar livro, Realizar empréstimo, Renovar empréstimo, Devolver livro, Registrar obra e Gerenciar usuários. Limite do sistema deve conter os casos. Um include razoável: "Realizar empréstimo" inclui "Verificar disponibilidade", pois a checagem sempre ocorre.

42. Descrição de “Realizar empréstimo”

Ator: Leitor. Pré-condições: leitor identificado e livro cadastrado. Fluxo: selecionar livro → solicitar empréstimo → sistema verifica disponibilidade e situação do leitor → registra empréstimo → informa prazo. Alternativa: livro indisponível; o sistema informa e oferece reserva, conforme regra definida. Pós-condições: empréstimo registrado e estado do livro atualizado para Emprestado.

43. Atividades

O desenho precisa ter nó inicial, ação “Selecionar livro”, ação “Consultar disponibilidade”, losango “Disponível?”, ramo [sim] para “Registrar empréstimo” e “Confirmar ao usuário”, ramo [não] para “Informar indisponibilidade”, e nó final. Cada ramo precisa estar ligado corretamente.

44. Classes e multiplicidades

Exemplo: Leitor(id,nome), Livro(isbn,titulo), Emprestimo(dataInicio,dataPrevista), Bibliotecario(id,nome). Um Leitor 1 — 0..* Emprestimos; cada empréstimo pertence a 1 leitor. Um Livro 1 — 0..* Emprestimos históricos; cada empréstimo refere-se a 1 livro. A multiplicidade 0..* permite que um leitor ainda não tenha empréstimos ou tenha vários ao longo do tempo.

45. Objetos

Exemplo: ana:Leitor nome="Ana"; livroA:Livro isbn="978..." titulo="Engenharia de Software"; emprestimo01:Emprestimo status="Ativo" dataInicio="02/11/2026". Vínculos: emprestimo01 ligado a ana e a livroA. Valores são concretos, não atributos genéricos.

46. Sequência

Participantes: Usuário, Portal, Serviço de Empréstimo, Repositório. Mensagens: solicitarEmprestimo → verificarDisponibilidade → consultarLivro → retorno de status. Fragmento alt: [disponível] registrarEmpréstimo e retornar confirmação; [indisponível] retornar recusa/informação. Ordem de cima para baixo e linhas de vida para todos os participantes.

47. Máquina de estados

Começa em Disponível. reservar → Reservado; retirar → Emprestado; devolver em bom estado → Disponível; devolver danificado → Em Manutenção; liberar após reparo → Disponível. As transições devem indicar eventos e, quando necessário, guardas.

48. Compra online

(a) Casos de uso para objetivos do cliente; (b) Atividades para fluxo de compra e decisões; (c) Sequência para mensagens entre cliente, aplicativo, estoque e pagamento; (d) Máquina de estados para o ciclo de vida do pedido: Aberto → Pago → Em Entrega → Entregue, com alternativas como Cancelado/Recusado conforme regras. Justificar pelo foco de cada desenho.

49. Componentes e implantação

Componentes: Navegador/Front-end, Serviço de Pedidos, Serviço de Estoque, Serviço de Pagamento, Persistência/Repository e suas interfaces/dependências. Implantação: dispositivo/navegador conecta via HTTPS ao servidor web; servidor web encaminha ao servidor de aplicação; servidor de aplicação acessa banco de dados. Os desenhos diferem porque um mostra módulos/contratos lógicos e o outro mostra nós físicos/ambientes e caminhos de comunicação.

50. Regra de empréstimo único

A regra aparece na descrição de requisitos/regras de negócio; no diagrama de casos de uso/descrição do fluxo como condição de solicitação; no diagrama de sequência na verificação antes do registro; e pode ser complementada por restrição do modelo. Teste: com um empréstimo ativo para um livro, enviar uma segunda solicitação para o mesmo livro deve ser recusada; o sistema não pode criar dois empréstimos ativos.

14. Checklist final para a véspera da prova

- ☐ Consigo explicar a diferença entre Engenharia de Software, software, sistema e arquitetura?
- ☐ Sei comparar cascata, prototipação, espiral/incremental e as quatro fases do RUP?
- ☐ Consigo classificar requisitos funcionais e não funcionais e melhorar requisitos vagos?
- ☐ Sei explicar UML x metodologia e modelo x diagrama?
- ☐ Consigo escolher o diagrama certo a partir da pergunta do enunciado?
- ☐ Sei desenhar caso de uso com ator, limite, associação, include e extend?
- ☐ Sei desenhar atividade com início, ações, decisão, guardas, fork/join e fim?
- ☐ Sei desenhar classe com atributos, operações, visibilidade, relações e multiplicidades?
- ☐ Consigo diferenciar classe e objeto?
- ☐ Sei ler sequência de cima para baixo e usar alt/opt/loop/par?
- ☐ Sei representar estados e transições com evento, guarda e efeito?
- ☐ Consigo explicar componente x implantação e relacionar modelos com testes?
- ☐ Sei diferenciar desenvolvimento incremental de iterativo?
- ☐ Lembro os quatro valores do Manifesto Ágil e os três pilares do Scrum?
- ☐ Sei explicar Product Owner, Scrum Master, Developers, Sprint, Daily, Review e Retrospective?
- ☐ Sei diferenciar Product Backlog, Sprint Backlog e Incremento?
- ☐ Consigo explicar Kanban, WIP, XP, TDD, refatoração, RAD e FDD?

Conselho final

Não tente decorar o desenho inteiro. Decore a pergunta que cada diagrama responde. Quando você entende a pergunta, fica muito mais fácil decidir o que entra no desenho.

Referências e origem do conteúdo

Fonte principal: PDF “ilovepdf_merged.pdf”, enviado na conversa (460 páginas), com conteúdos de Engenharia de Software, Engenharia de Requisitos, Arquitetura e Modelagem de Sistemas com UML. Os números abaixo são as páginas do arquivo PDF:

- Fundamentos, visão sistêmica, software e arquitetura inicial: pp. 1–35.
- Métodos, ferramentas, construção, usabilidade e princípios: pp. 69–97.
- Ciclos de vida e RUP: pp. 98–143.

- Requisitos e arquitetura: pp. 158–170.
- Técnicas de levantamento: pp. 240–255.
- Requisitos funcionais/não funcionais, documento, processo e viabilidade: pp. 256–275.
- Modelagem, UML e 14 diagramas: pp. 276–350.
- Métodos ágeis, Manifesto Ágil, Scrum, Kanban, XP/TDD, RAD e FDD: pp. 415–460.
- Manifesto Ágil: https://agilemanifesto.org/iso/ptbr/manifesto.html; Princípios: https://agilemanifesto.org/iso/ptbr/principles.html.
- Scrum Guide (referência indicada no PDF): https://scrumguides.org/; Kanban Guide (referência indicada no PDF): https://kanban.university/kanban-guide.

Complementos consultados para indicar vídeos e materiais adicionais:

DevMedia — Aula demonstrativa de UML — https://www.youtube.com/watch?v=2dSq0Vu1GFo

Prof. Gilleanes Guedes — Diagrama de Casos de Uso — https://www.youtube.com/watch?v=qVemjrDKw2k

Gabriel Skrilatt — Modelagem UML — https://www.youtube.com/watch?v=pCCqiJDLr4Y

DevMedia — Curso de Diagrama de Classes — https://www.devmedia.com.br/curso/introducao-a-construcao-de-diagrama-de-classes-da-uml/307

e-Aulas USP — Diagrama de Classes — https://eaulas.usp.br/portal/video?idItem=27545

Prof. Gilleanes Guedes — Diagrama de Atividades, Parte III — https://www.youtube.com/watch?v=HBJvuhkX-Vs

Eduardo Engenharia de Software — Diagrama de Sequência — https://www.youtube.com/watch?v=6khudydsYYQ

Prof. Gilleanes Guedes — Sequência: exemplos — https://www.youtube.com/watch?v=Xzatg5Fz2eQ

Fabio de Lima Bezerra — Máquina de Estados — https://www.youtube.com/watch?v=N5gfaNcdYjI

Scrum.org — Scrum em 21 minutos — https://www.scrum.org/resources/blog/scrum-em-21-minutos

Celso Kitamura — O que é Kanban? — https://www.youtube.com/watch?v=MuIW3__Srh0

Economia para Iniciantes — Kanban na prática — https://www.youtube.com/watch?v=CqA5XKlraOk

Coursera — TDD: Desenvolvimento de Software Guiado por Testes — https://www.coursera.org/learn/tdd-desenvolvimento-de-software-guiado-por-testes

Observação: as analogias, exemplos didáticos e exercícios foram elaborados para facilitar a aprendizagem. Quando o exemplo não está literalmente no PDF, ele está identificado como exemplo complementar; as regras e a terminologia centrais seguem o material da disciplina.